

Computational Economics

Lab: Introduction to Git

Version control, project history, and collaboration

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Three Cases

- ▶ **Case: Keep earlier versions**

You keep revising a file but worry that saving your latest edits will erase useful earlier work. Soon you have v1, v2, v3, ... What changed, and which version should you use?

With Git: record versions in one project and revisit earlier work.

- ▶ **Case: Work with others**

You and a partner edit the same project. You exchange files and explain each change in messages or meetings, then try to combine your edits without losing either person's work.

With Git: compare changes, see who made them, and merge contributions.

- ▶ **Case: Build on an existing project**

You find a useful project online and want to try it or adapt it. After downloading a copy, its author releases improvements. How do you bring those updates into your own copy?

With Git: clone the repository and fetch later updates to integrate.

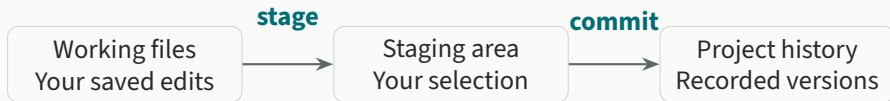
Git: Start with a Working Folder

Git is software you install on your computer to keep a history of your work. It helps you record versions, revisit earlier work, and combine changes.

1. Start with an ordinary working folder containing your files.
2. **Initialize** Git in that folder (`git init`). The folder becomes a **repository**: a project whose history Git can record.
3. Keep editing your files as usual. Choose which changes to record and when to record them; initializing alone does not save a first version.

To start from an existing project, **clone** its repository: make a local copy of its files and history instead of initializing an empty one.

Git: Record Your Changes



- ▶ Edit and save a file as usual. Use **status** to see which files have changed and **diff** to inspect the changes.
- ▶ **Stage**: select the changes you want to include in the next recorded version.
- ▶ **Commit**: record that selection in the project's history, with a short message explaining what changed.
- ▶ Use the **log** to browse earlier commits. You can compare versions and recover earlier content when needed.

Saving a file updates your working copy; committing records a version in Git.

Git: Combine and Share Changes

- ▶ A **branch** lets you develop a change separately, such as trying a new idea while keeping the main version available.
- ▶ **Merge**: bring changes from another branch into your current branch. If edits clash, Git flags a **conflict**; you decide how to combine them.
- ▶ A **remote** is a saved connection to another copy of the repository, for example one shared with your collaborators online.
- ▶ **Push**: send your local commits to that remote so others can access them.
- ▶ **Pull**: get changes from the remote and combine them with your current branch. Conflicts may need your attention.

Commits stay on your computer until you share them. Git can record your work even when you are offline.

What Is GitHub?

GitHub is an online platform that hosts Git repositories and provides tools for sharing projects and collaborating on them.

- ▶ Upload a repository to make its files and history available to collaborators, or publish it for anyone to access.
- ▶ Browse changes, discuss tasks through issues, and review proposed contributions through pull requests.
- ▶ Connect your local repository to the hosted one through a **remote**: a named connection to another repository.
- ▶ Exchange changes with that remote, or **clone** an existing repository to start working with a local copy.

One Small Change, One Clear Commit

Case: Improve the README

1. Open the project folder in VS Code and inspect its Source Control panel.
2. Add a sentence explaining how to run the program.
3. Inspect the **diff**: the comparison between old and new content.
4. Follow the new instructions to check that they work.
5. Stage the intended files and commit with a useful message, such as “Add instructions for running the program.”

Before committing, check that the diff contains only the changes you intended.

Choose What Belongs in the Repository

Track and share

Source code: `.py` and `.ipynb`
A README explaining how to run the work
A list of required packages
Small, relevant results when requested

Keep outside the shared history

Local environment: `.venv/`
Caches and notebook checkpoints
Passwords, access tokens, and private data
Large generated files that can be recreated

- ▶ A **.gitignore** file lists patterns for files Git should leave untracked.
- ▶ Ignoring an already tracked file does not remove it from the repository or its history.
- ▶ Check notebook outputs as well as code before pushing. A public repository makes its tracked content accessible to others.

Connect Your Folder to GitHub

1. Install Git, register for GitHub, and open your project folder in VS Code.
2. In Source Control, initialize the folder as a Git repository. Set your commit name and email when prompted.
3. Add a README and a `.gitignore`; inspect, stage, and commit them.
4. Choose **Publish to GitHub**, sign in through the browser, choose a repository name and visibility, and check which files are included.
5. Open the resulting GitHub URL and confirm that your commit and files appear.

This workflow creates the GitHub repository and connects the local folder. If you already have a GitHub repository, clone it first and work in that local folder.

Practice: Is the Change on GitHub Yet?

Case: A Missing README Update

You edit and save the README, make a local commit, and open GitHub. The website still shows the old README.

- ▶ Which steps have completed: save, stage, commit, push?
- ▶ What would you check before trying again?
- ▶ After pushing, where can you verify the commit message and changed file?

Repeat with a second small edit. Explain the difference between the local history and the remote history to a partner.

References and Further Reading

1. **Prof. Feng Zhigang (2026), AI for Economic Research: Dynamic Models, Language, and Agents. [Online course]**
 - ▶ Session 1.5: The Workbench: Python, VS Code, Git and GitHub
2. **Microsoft, Visual Studio Code Documentation. [Webpages]**
 - ▶ Quickstart: use source control in VS Code
3. **GitHub, GitHub Docs. [Webpage]**
 - ▶ Creating an account on GitHub
4. **Scott Chacon and Ben Straub, Pro Git. 2nd ed. [Book]**
 - ▶ Installing Git